

Object Oriented

Programming (CS1004)

Date: May 27th 2024

Course Instructor(s)

Dr. Sajid Anwer, Mr. Usman Ghous, Mr. Asif
Ameer, Mr. Rizwan Ul Haq, Mr. Jawad Khalid,
Mr. Muiz Qadir

Final Term Exam

Total Time (Min): 140

Total Marks: 75

Total Questions: 6

Invigilator's Signature

Roll No

Section

Student Signature

- This subjective part is to be solved on the answer sheets provided.
- This subjective part can only be given to the student upon returning the objective part any time before end of objective part time.
- Calculators are not allowed

<Subjective Part>

CLO 1: Demonstrate the basic concepts of OOP.

[10 marks]

Q3. Imagine a tollbooth at a bridge. Cars passing by the booth are expected to pay a 50 Rupee toll, the toll amount cannot be changed. Mostly they do, but sometimes a car goes by without paying. The tollbooth keeps track of the number of cars that have gone by and of the total amount of money collected. Model this tollbooth with a class called tollBooth.

- ✓ The two data items hold the total number of cars, and the total amount of money collected.
- ✓ Initializes both of these with default data for example 0 for both total cars and total amount.
- Whenever a vehicle passes, increment the car total and add the amount of 50 to the total amount. If a car passes without payment, then increments the cars total but adds nothing to the cash total.
- Finally, a member function called display() displays the total number of cars passed and the total amount collected.
- In execution, your program must show one car passes by paying a toll and one without paying a toll.

National University of Computer and Emerging Sciences

Chiniot-Faisalabad Campus

CLO 2: Apply OOP concepts (Encapsulation, Inheritance, Polymorphism, Abstraction) to computing problems for the related program. **[10 marks]**

Q4. Complete the missing parts of the class that need to be there, for the given code to work properly. Assume headers included.

```
class Matrix {
private:
    int rows, cols;
    int** data;
public:
    Matrix(int r, int c) : rows(r), cols(c) {
        data = new int*[rows];
        for (int i = 0; i < rows; ++i) {
            data[i] = new int[cols];
            //Initializes complete row to 0
            memset(data[i], 0, cols * sizeof(int));
        }
    }
    ~Matrix() {
        for (int i = 0; i < rows; ++i) {
            delete[] data[i];
        }
        delete[] data;
    }
    void setValue(int r, int c, int value) {
        if (r >= 0 && r < rows && c >= 0 && c < cols) {
            data[r][c] = value;
        }
    }
    //Copy Constructor (deep copy)
    //Overloaded assignment operator (deep copy)
    //Overloaded insertion operator
};

int main() {
    Matrix mat1(3, 3);
    mat1.setValue(0, 0, 2);
    mat1.setValue(1, 1, 4);
    mat1.setValue(2, 2, 6);
    Matrix mat2 = mat1;
    cout << "Matrix 1:\n" << mat1;
    cout << "Matrix 2 (copied from Matrix 1):\n" << mat2;
    system("pause");
    return 0;
}
```

CLO 2: Apply OOP concepts (Encapsulation, Inheritance, Polymorphism, Abstraction) to computing problems for the related program. **[6+2+7 = 15 marks]**

Q5. The inheritance is powerful concept of OOP. And multiple inheritance can lead to ambiguity issues, especially with the **Diamond Problem**, where a derived class inherits from two base classes that both inherit from a common ancestor. To address these ambiguities, **two primary mechanisms** are commonly used:

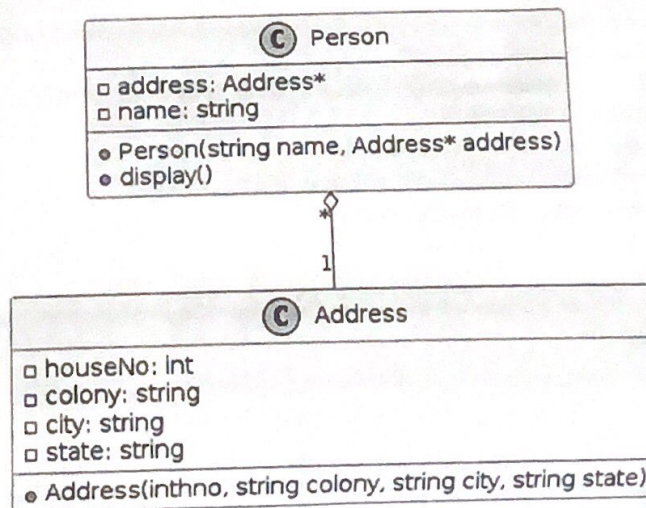
virtual

- a) Write down the correct versions of following code by employing both of these mechanisms. Each implementation will carry equal marks. (3+3)
- b) Provide a logical justification why these two implementations are provided to solve such ambiguities. You may aid your answer with schematic diagram. Irrelevant details will lead you ZERO marks. (2)

<pre> class Human { public: Human(const string&name):name(name) {} void introduce() { cout << "I am a human named " << name << endl; } private: string name; }; class Person :public Human { public: Person(const string& name):Human(name) {} void greet() { cout << "Hello I am a person." << } }; end; </pre>	<pre> class Employee:public Human { public: Employee(const string& name) :Human(name) {} void work() { cout << "I am working as an " << "employee." << endl; } }; class Manager :public Person, public Employee{ public: Manager(const string& name) :Person(name), Employee(name) {} }; int main(){ Manager manager("XYZ"); manager.introduce(); manager.greet(); manager.work(); } </pre>
--	---

- c) Demonstrate the **Aggregation** type relation between the classes by coding the following UML diagram. Squares and solid circles are representing private and public members of class respectively.

1. Write a Person class (3)
2. Writing Address class (3)
3. Driver code (should have objects of both classes) and demonstration relationship (1)



National University of Computer and Emerging Sciences
Chiniot-Faisalabad Campus

CLO 3: Model an algorithmic solution for a given problem using OOP [8+3+4 = 15 marks]

Q6. Implementing a discount management module for an e-commerce system developed in c++. You already have a struct and a class for **CartItem** and **Cart** respectively. You can assume both of these are correctly implemented with different types of constructors, getters and setters.

- Struct with the name **CartItem**
 - Attributes
 - **Item_name**: string, **item_unit_price**: int, **item_quantity**: int
 - Methods
 - Constructors, getter and setter as required
- Class with the name **Cart**
 - Attributes
 - **CartItem**: pointer of CartItem, **subTotal**: int, **itemCount**: int, **discountClass**: pointer of DiscountBase
 - Methods
 - Constructors, getter and setter as required

Your implementation requirements start here:

- Design and Implement the **DiscountBase** parent class and its child classes to implement following three discount rules (2+3+3)
 - If itemCount in cart is less than 50 and subtotal is greater than 10,000 Rs then apply fixed price discount of 1000 Rs
 - If itemCount in cart is greater than 50 but less than 100, and subtotal is greater than 10,000 Rs then apply percentage discount of 5%
 - If itemCount in cart is greater than greater than 100, and subtotal is greater than 10,000 Rs then apply percentage discount of 10%

Implement the above logic for each class in the function **float discountAmount(subTotal, itemCount)**. Apply polymorphism concepts as required.
- The class **Cart** also has the following methods. Implement these functions (1+2)
 - **void addToCart(CartItem itemToAdd)**
 - Adds **itemToAdd** to the **CartItem** pointer array
 - Updates the **subTotal** As per item's unit price and item's quantity
 - Updates the **itemCount** As per item's quantity
 - **float totalPrice()**
 - Returns the total cart price by subtracting discounted price from subtotal
 - This function calls the **discountAmount(subTotal, itemCount)** function of base class **DiscountBase**.
- Implement a main function that properly tests all three rules and respective polymorphic implementation (4)

Make use of

- one array of CartItem
- one object of Cart
- one object of each child class of DiscountBase Class
- Gradually increase items in the cart to meet each discount rule one at a time.
- Associate correct child class of DiscountBase to the Cart object and print Total using totalPrice function of Cart Class

CLO 4: Apply good programming practices

[10 marks]

Q7. Implement a String Conversion Utility with Robust Exception Handling

You are tasked with creating a utility class called `Converter` that can perform string-to-integer and string-to-float conversions. The class should handle various types of exceptions that may occur during the conversion process.

Specifically, you are required to implement the following:

stringToInt method:

- This method should take a **std::string** as input and attempt to convert it to an integer value.
- If the input string is in an invalid integer format (e.g., contains non-numeric characters), the method should throw a **std::invalid_argument** exception with an appropriate error message.
- If the resulting integer value is out of the range of the int data type (-2,147,483,647 to 2,147,483,647), the method should throw a **std::out_of_range** exception with an appropriate error message.

stringToFloat method:

- This method should take a **std::string** as input and attempt to convert it to a floating-point value.
- If the input string is in an invalid float format (e.g., contains non-numeric characters), the method should throw a **std::invalid_argument** exception with an appropriate error message.

main function:

- In the main function, you should test the **stringToInt** and **stringToFloat** methods with various input strings, including valid and invalid formats.
- For each conversion attempt, you should handle the possible exceptions (**std::invalid_argument** and **std::out_of_range**) and print appropriate error messages.

Note: Your solution should include the implementation of the `Converter` class and the main function, where you test the conversion methods and handle the exceptions accordingly.

National University of Computer and Emerging Sciences
Chiniot-Faisalabad Campus

CLO 4: Apply good programming practices

[15 marks]

Q8. You are required to implement a generic container class and a set of utility functions that can work with this container. The container should be able to store and manipulate data of different data types, and the utility functions should be able to operate on the container regardless of the data type.

GenericContainer template class:

Implement a **template class** called **GenericContainer** that can store elements of any data type.

The class should have the following member functions:

- **void add(T element):** Adds an element of type T to the container.
- **T get(int index):** Returns the element at the specified index in the container.
- **int size():** Returns the number of elements currently stored in the container.

Template Functions (Global functions):

Implement the following **template functions** that can work with the **GenericContainer** class:

- **template <typename T>**
T sum(const GenericContainer<T>& container): Returns the sum of all elements in the container.
- **template <typename T, typename U, typename V>**
GenericContainer<V> merge(const GenericContainer<T>& container1, const GenericContainer<U>& container2): Creates a new **GenericContainer** of type V by merging the elements from the two input containers.
- **template <typename T>**
void print(const GenericContainer<T>& container): Prints all the elements in the container.

Test Cases:

In the main function, create instances of the **GenericContainer** class with different data types (e.g., int, float, std::string) and test the functionality of the class and the utility functions. Verify that the **sum**, **merge**, and **print** functions work correctly with the **GenericContainer** instances. Test the behavior of the functions when the container holds elements of different data types.

Note: Your solution should include the implementation of the **GenericContainer** class and the template functions, as well as the main function where you test the functionality of the provided components.